

Федеральное государственное автономное образовательное учреждение высшего
образования «Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №7
по дисциплине «Методы Оптимизации»

Вариант: 8

Преподаватель:
Селина Елена Георгиевна

Выполнили:
Девярых Павел Леонидович
Караганов Павел Эдуардович
Полуянов Игорь Андреевич
Группа: P3210

Санкт-Петербург, 2026 г.

Содержание

1	Задание 1. Аналитическое исследование функции (3 балла)	2
1.1	Постановка задачи	2
1.2	Область определения	2
1.3	Поиск стационарных точек	2
1.4	Матрица Гессе и определение типа стационарной точки	3
1.5	Собственные значения матрицы Гессе	3
1.6	Поиск глобального минимума на границе	3
1.6.1	Левая граница: $x = -20, y \in [-50, 50]$	4
1.6.2	Правая граница: $x = 20, y \in [-50, 50]$	4
1.6.3	Верхняя граница: $y = 50, x \in [-20, 20]$	4
1.6.4	Нижняя граница: $y = -50, x \in [-20, 20]$	4
1.6.5	Сравнение значений на границах	5
1.7	Линии уровня	5
1.8	Выводы по заданию 1	5
2	Задание 2. Градиентный спуск и эвристики (5 баллов)	7
2.1	Метод по варианту	7
2.1.1	Общая формула градиентного спуска	7
2.1.2	Формула Adagrad	7
2.2	Пункт 2.1. Градиентный спуск, сходящийся к седловой точке	7
2.2.1	Параметры	7
2.2.2	Формулы обновления	7
2.2.3	Первые итерации (проверка)	8
2.2.4	Результат	8
2.3	Пункт 2.2. Собственная реализация Adagrad	9
2.3.1	Реализация	9
2.3.2	Почему Adagrad преодолевает седловую точку	9
2.4	Пункт 2.3. Подбор гиперпараметров: пилообразная и плавная траектории . .	10
2.4.1	Пилообразная траектория ($\alpha = 22,0$)	10
2.4.2	Плавная траектория ($\alpha = 5,0$)	10
2.4.3	Сравнение	11
2.5	Пункт 2.4. Встроенная реализация <code>torch.optim.Adagrad</code>	11
2.6	Выводы по заданию 2	12
3	Задание 3. Нейросеть с кастомизированным оптимизатором (3 балла)	14
3.1	Описание задачи и датасета	14
3.2	Целевая функция: Binary Cross-Entropy Loss	14
3.3	Архитектура нейросети	14
3.4	Кастомная реализация Adagrad как PyTorch-оптимизатор	15
3.5	Обучение и результаты	15
3.5.1	Кривые обучения	16
3.5.2	Итоговые результаты	16
3.6	Анализ результатов	16
3.7	Выводы по заданию 3	17
4	Общие выводы	18

1 Задание 1. Аналитическое исследование функции (3 балла)

1.1 Постановка задачи

Дана функция по варианту 8:

$$f(x, y) = 10^{-2}(8x^2 + 2xy + 27x + 6y + 9) \quad (1)$$

в области $(x, y) \in [-20, 20] \times [-50, 50]$.

Требуется:

1. Найти все стационарные точки, определить их тип (седловые точки, локальные экстремумы).
2. Найти глобальный минимум.
3. Построить линии уровня функции.
4. Сделать выводы о сложностях для оптимизационных алгоритмов.

1.2 Область определения

Без ограничений область определения функции: ООФ $f(x, y) = \mathbb{R}^2$.

Частные производные первого порядка существуют в каждой точке области определения (функция — многочлен).

Область определения с учётом ограничения задачи:

$$(x, y) \in [-20, 20] \times [-50, 50].$$

1.3 Поиск стационарных точек

Вычислим частные производные первого порядка:

$$\frac{\partial f}{\partial x} = 10^{-2}(16x + 2y + 27), \quad (2)$$

$$\frac{\partial f}{\partial y} = 10^{-2}(2x + 6). \quad (3)$$

Приравняем к нулю:

$$\begin{cases} \frac{\partial f}{\partial x} = 0 \\ \frac{\partial f}{\partial y} = 0 \end{cases} \implies \begin{cases} 10^{-2}(16x + 2y + 27) = 0 \\ 10^{-2}(2x + 6) = 0 \end{cases} \implies \begin{cases} 16x + 2y + 27 = 0 \\ 2x + 6 = 0 \end{cases}$$

Из второго уравнения:

$$2x + 6 = 0 \implies \boxed{x = -3}$$

Подставляем в первое уравнение:

$$16 \cdot (-3) + 2y + 27 = 0 \implies -48 + 2y + 27 = 0 \implies 2y = 21 \implies \boxed{y = 10,5}$$

Единственная стационарная точка: $M(-3; 10,5)$.

1.4 Матрица Гессе и определение типа стационарной точки

Вычислим частные производные второго порядка:

$$f_{xx} = \frac{\partial^2 f}{\partial x^2} = 10^{-2} \cdot 16 = 0,16, \quad (4)$$

$$f_{yy} = \frac{\partial^2 f}{\partial y^2} = 10^{-2} \cdot 0 = 0, \quad (5)$$

$$f_{xy} = f_{yx} = \frac{\partial^2 f}{\partial x \partial y} = 10^{-2} \cdot 2 = 0,02. \quad (6)$$

Составим матрицу Гессе:

$$H = \begin{pmatrix} f_{xx} & f_{xy} \\ f_{yx} & f_{yy} \end{pmatrix} = 10^{-2} \cdot \begin{pmatrix} 16 & 2 \\ 2 & 0 \end{pmatrix} = \begin{pmatrix} 0,16 & 0,02 \\ 0,02 & 0 \end{pmatrix}$$

Вычислим угловые миноры:

$$D_1 = f_{xx} = 0,16 > 0, \quad (7)$$

$$D_2 = \det(H) = f_{xx} \cdot f_{yy} - f_{xy}^2 = 0,16 \cdot 0 - (0,02)^2 = -0,0004. \quad (8)$$

$$\boxed{D_2 = -0,0004 < 0}$$

Так как $D_2 < 0$, гессиан неопределён, следовательно, точка $M(-3; 10,5)$ является **седловой точкой**.

Локальных экстремумов (максимумов и минимумов) внутри области нет, поскольку единственная стационарная точка — седловая.

1.5 Собственные значения матрицы Гессе

Для более глубокого понимания найдём собственные значения H :

$$\det(H - \lambda I) = 0 \implies \begin{vmatrix} 0,16 - \lambda & 0,02 \\ 0,02 & -\lambda \end{vmatrix} = 0$$

$$(0,16 - \lambda)(-\lambda) - (0,02)^2 = 0 \implies \lambda^2 - 0,16\lambda - 0,0004 = 0$$

По формуле дискриминанта:

$$\lambda_{1,2} = \frac{0,16 \pm \sqrt{0,0256 + 0,0016}}{2} = \frac{0,16 \pm \sqrt{0,0272}}{2} = \frac{0,16 \pm 0,1650}{2}$$

$$\lambda_1 \approx 0,1625 > 0, \quad \lambda_2 \approx -0,0025 < 0.$$

Собственные значения имеют **разные знаки**, что подтверждает седловой характер точки. Направление, соответствующее $\lambda_2 < 0$ — это направление, вдоль которого функция убывает (примерно вдоль оси y). Именно в этом направлении градиент очень мал, что создаёт сложности для градиентного спуска.

1.6 Поиск глобального минимума на границе

Поскольку внутри области нет локальных минимумов, глобальный минимум достигается на границе прямоугольника $[-20, 20] \times [-50, 50]$. Исследуем все четыре стороны.

1.6.1 Левая граница: $x = -20, y \in [-50, 50]$

$$\begin{aligned} f(-20, y) &= 10^{-2}(8 \cdot 400 + 2 \cdot (-20)y + 27 \cdot (-20) + 6y + 9) \\ &= 10^{-2}(3200 - 40y - 540 + 6y + 9) \\ &= 10^{-2}(2669 - 34y). \end{aligned}$$

Функция линейна по y с коэффициентом $-34 < 0$, т.е. убывает. Минимум при $y = 50$:

$$f(-20, 50) = 10^{-2}(2669 - 1700) = 10^{-2} \cdot 969 = 9,69.$$

1.6.2 Правая граница: $x = 20, y \in [-50, 50]$

$$\begin{aligned} f(20, y) &= 10^{-2}(8 \cdot 400 + 2 \cdot 20 \cdot y + 27 \cdot 20 + 6y + 9) \\ &= 10^{-2}(3200 + 40y + 540 + 6y + 9) \\ &= 10^{-2}(3749 + 46y). \end{aligned}$$

Функция линейна по y с коэффициентом $46 > 0$, т.е. возрастает. Минимум при $y = -50$:

$$f(20, -50) = 10^{-2}(3749 - 2300) = 10^{-2} \cdot 1449 = 14,49.$$

1.6.3 Верхняя граница: $y = 50, x \in [-20, 20]$

$$\begin{aligned} f(x, 50) &= 10^{-2}(8x^2 + 2x \cdot 50 + 27x + 6 \cdot 50 + 9) \\ &= 10^{-2}(8x^2 + 100x + 27x + 300 + 9) \\ &= 10^{-2}(8x^2 + 127x + 309). \end{aligned}$$

Это парабола по x с $a = 8 > 0$ (ветви вверх). Вершина:

$$x^* = -\frac{127}{2 \cdot 8} = -\frac{127}{16} = -7,9375.$$

Точка $x^* = -7,9375 \in [-20, 20]$ — попадает в интервал. Минимум:

$$\begin{aligned} f(-7,9375, 50) &= 10^{-2}(8 \cdot (-7,9375)^2 + 127 \cdot (-7,9375) + 309) \\ &= 10^{-2}(8 \cdot 63,004 - 1008,06 + 309) \\ &= 10^{-2}(504,03 - 1008,06 + 309) \\ &= 10^{-2} \cdot (-195,03) \approx -1,9503. \end{aligned}$$

1.6.4 Нижняя граница: $y = -50, x \in [-20, 20]$

$$\begin{aligned} f(x, -50) &= 10^{-2}(8x^2 + 2x \cdot (-50) + 27x + 6 \cdot (-50) + 9) \\ &= 10^{-2}(8x^2 - 100x + 27x - 300 + 9) \\ &= 10^{-2}(8x^2 - 73x - 291). \end{aligned}$$

Парабола по x с $a = 8 > 0$. Вершина:

$$x^* = \frac{73}{2 \cdot 8} = \frac{73}{16} = 4,5625.$$

Точка $x^* = 4,5625 \in [-20, 20]$ — попадает в интервал. Минимум:

$$\begin{aligned} f(4,5625, -50) &= 10^{-2}(8 \cdot (4,5625)^2 - 73 \cdot 4,5625 - 291) \\ &= 10^{-2}(8 \cdot 20,8164 - 333,0625 - 291) \\ &= 10^{-2}(166,5313 - 333,0625 - 291) \\ &= 10^{-2} \cdot (-457,5313) \approx -4,5753. \end{aligned}$$

1.6.5 Сравнение значений на границах

Граница	Точка минимума	$f(x, y)$
Левая ($x = -20$)	$(-20, 50)$	9,69
Правая ($x = 20$)	$(20, -50)$	14,49
Верхняя ($y = 50$)	$(-7,9375, 50)$	-1,9503
Нижняя ($y = -50$)	$(4,5625, -50)$	-4,5753

Таблица 1: Минимальные значения функции на каждой стороне границы

Глобальный минимум достигается на нижней границе в точке $(4,5625; -50)$ и равен:

$$f_{\min} = f(4,5625; -50) \approx -4,5753.$$

1.7 Линии уровня

На рис. 1 представлены линии уровня функции $f(x, y)$ с отмеченными особыми точками.

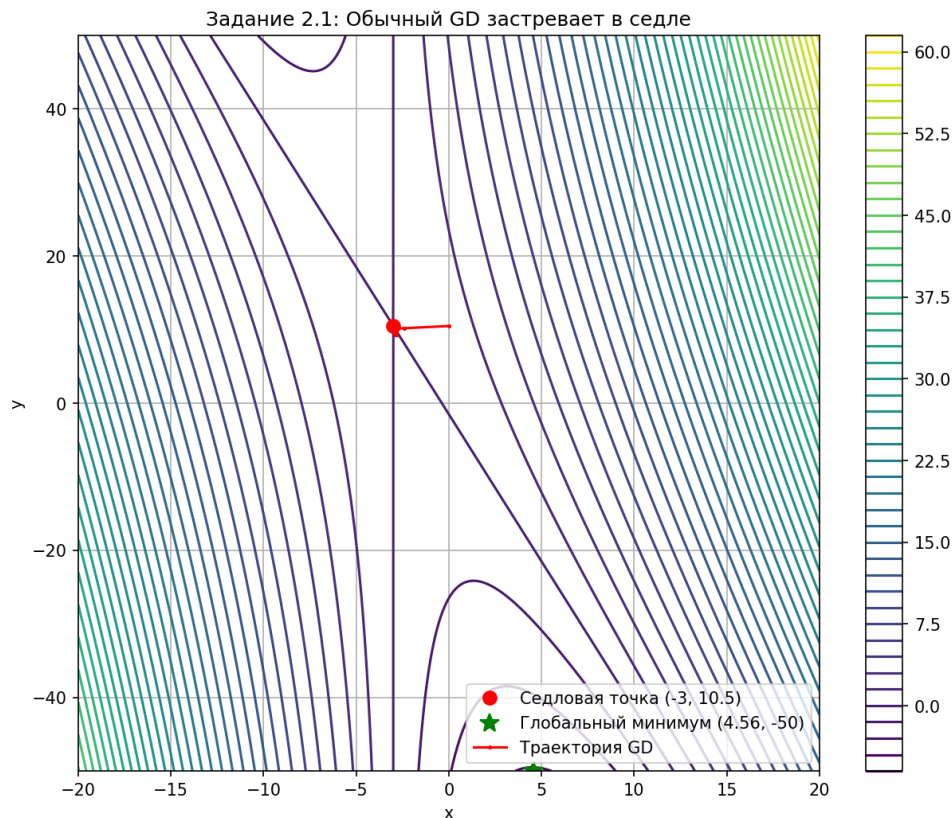


Рис. 1: Линии уровня функции $f(x, y)$. Красная точка — седловая точка $(-3; 10,5)$, зелёная звезда — глобальный минимум $(4,5625; -50)$.

1.8 Выводы по заданию 1

1. Функция имеет единственную стационарную точку $M(-3; 10,5)$, которая является **седловой** ($\det H = -0,0004 < 0$).

2. Локальных экстремумов нет.

3. Глобальный минимум $f \approx -4,5753$ достигается на границе области в точке $(4,5625; -50)$.

4. Сложности для оптимизационных алгоритмов:

- Классический градиентный спуск может «застрять» в окрестности седловой точки, поскольку градиент в ней равен нулю.
- Собственное значение $\lambda_2 \approx -0,0025$ очень мало по модулю — это означает, что вдоль соответствующего направления (примерно вдоль y) функция меняется крайне медленно, создавая эффект «плато».
- Необходимы адаптивные методы (Adagrad, RMSProp, Adam) для преодоления таких участков.

2 Задание 2. Градиентный спуск и эвристики (5 баллов)

2.1 Метод по варианту

Вариант 8 соответствует методу **Adagrad** (варианты 2, 5, 8, 11, 14, 17).

2.1.1 Общая формула градиентного спуска

Классический градиентный спуск обновляет параметры по правилу:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \cdot \nabla f(\theta^{(t)}), \quad (9)$$

где α — скорость обучения (learning rate), ∇f — градиент функции.

2.1.2 Формула Adagrad

Метод Adagrad [?] адаптивно подстраивает скорость обучения для каждого параметра:

$$\begin{aligned} G_i^{(t)} &= G_i^{(t-1)} + \left(\frac{\partial f}{\partial \theta_i} \right)^2, \\ \theta_i^{(t+1)} &= \theta_i^{(t)} - \frac{\alpha}{\sqrt{G_i^{(t)} + \varepsilon}} \cdot \frac{\partial f}{\partial \theta_i}, \end{aligned} \quad (10)$$

где:

- $G_i^{(t)}$ — накопленная сумма квадратов градиентов по i -му параметру за все итерации;
- α — начальная скорость обучения (гиперпараметр);
- ε — малая константа для числовой устойчивости ($\varepsilon = 10^{-8}$).

Ключевая идея: если по некоторому параметру градиенты были малы (малый G_i), то $\frac{\alpha}{\sqrt{G_i}}$ будет большим — шаг увеличивается. Это позволяет «преодолевать» области с малым градиентом (например, вблизи седловых точек).

2.2 Пункт 2.1. Градиентный спуск, сходящийся к седловой точке

2.2.1 Параметры

- Начальное приближение: $(x_0, y_0) = (0, 0; 10, 5)$.
- Скорость обучения: $\alpha = 5, 0$.
- Количество итераций: $N = 100$.
- Критерий останова: фиксированное число итераций + контроль нормы градиента.

2.2.2 Формулы обновления

Для нашей функции градиент задаётся формулами (2)–(3):

$$\nabla f(x, y) = \begin{pmatrix} 10^{-2}(16x + 2y + 27) \\ 10^{-2}(2x + 6) \end{pmatrix}$$

Правило обновления:

$$x^{(t+1)} = x^{(t)} - 5, 0 \cdot 10^{-2}(16x^{(t)} + 2y^{(t)} + 27), \quad (11)$$

$$y^{(t+1)} = y^{(t)} - 5, 0 \cdot 10^{-2}(2x^{(t)} + 6). \quad (12)$$

2.2.3 Первые итерации (проверка)

Итерация 0 $\rightarrow$ 1: начальная точка $(0; 10,5)$:

$$\begin{aligned}g_x &= 0,01 \cdot (16 \cdot 0 + 2 \cdot 10,5 + 27) = 0,01 \cdot 48 = 0,48, \\g_y &= 0,01 \cdot (2 \cdot 0 + 6) = 0,06, \\x^{(1)} &= 0 - 5 \cdot 0,48 = -2,4, \\y^{(1)} &= 10,5 - 5 \cdot 0,06 = 10,2.\end{aligned}$$

Итерация 1 $\rightarrow$ 2: точка $(-2,4; 10,2)$:

$$\begin{aligned}g_x &= 0,01 \cdot (16 \cdot (-2,4) + 2 \cdot 10,2 + 27) = 0,01 \cdot (-38,4 + 20,4 + 27) = 0,09, \\g_y &= 0,01 \cdot (2 \cdot (-2,4) + 6) = 0,01 \cdot 1,2 = 0,012, \\x^{(2)} &= -2,4 - 5 \cdot 0,09 = -2,85, \\y^{(2)} &= 10,2 - 5 \cdot 0,012 = 10,14.\end{aligned}$$

Видно, что метод сходится к $(-3; 10,5)$, при этом градиент быстро уменьшается.

2.2.4 Результат

После 100 итераций:

- Конечная точка: $x = -2,8478$, $y = 9,2634$.
- Норма градиента: $\|\nabla f\| = 0,003068 \approx 0$.
- Значение функции: $f = -0,0019$.

Вывод: метод «застрял» в окрестности седловой точки $(-3; 10,5)$, т.к. градиент в ней стремится к нулю, и шаги становятся пренебрежимо малы.

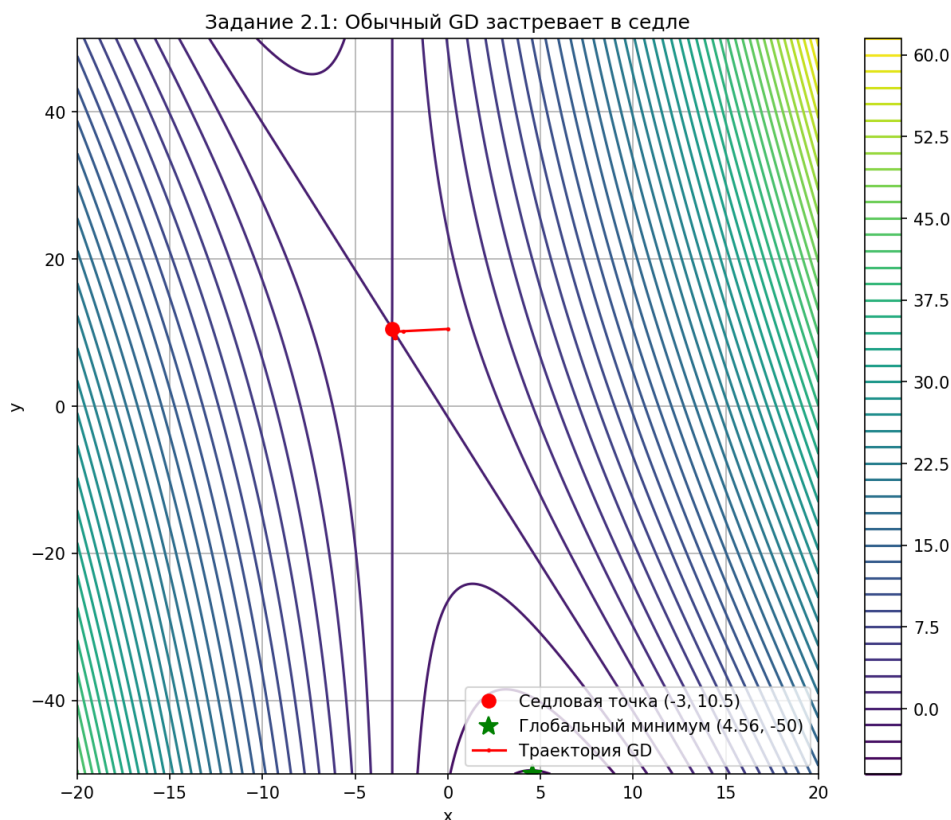


Рис. 2: Пункт 2.1. Траектория обычного GD: метод застревает вблизи седловой точки $(-3; 10,5)$.

2.3 Пункт 2.2. Собственная реализация Adagrad

2.3.1 Реализация

Для каждого параметра (x и y) заводим отдельный аккумулятор G :

Листинг 1: Реализация Adagrad (ручная)

```

1 G_x, G_y = 0.0, 0.0
2
3 for _ in range(100):
4     g = grad_f(x, y)
5     G_x += g[0]**2          # G_x = sum of squared gradients for x
6     G_y += g[1]**2          # G_y = sum of squared gradients for y
7     x = x - (lr / sqrt(G_x + eps)) * g[0]
8     y = y - (lr / sqrt(G_y + eps)) * g[1]
9     x = clip(x, -20, 20)
10    y = clip(y, -50, 50)

```

2.3.2 Почему Adagrad преодолевает седловую точку

В окрестности седловой точки $(-3; 10,5)$:

- Градиент по y : $\frac{\partial f}{\partial y} = 0,01(2x + 6)$. При $x \approx -3$ он ≈ 0 .
- Следовательно, G_y накапливается очень медленно, остаётся малым.
- Адаптивный lr : $\frac{\alpha}{\sqrt{G_y + \epsilon}} \rightarrow$ **очень большой**.

- Результат: даже крошечный градиент по y , умноженный на огромный адаптивный lr , даёт достаточный шаг для «выхода» из окрестности седла.

По сути, Adagrad компенсирует малые градиенты большим адаптивным шагом.

2.4 Пункт 2.3. Подбор гиперпараметров: пилообразная и плавная траектории

2.4.1 Пилообразная траектория ($\alpha = 22,0$)

При большом значении α начальный эффективный шаг $\frac{\alpha}{\sqrt{G+\varepsilon}}$ получается очень большим. Метод «перелетает» минимум, затем корректирует, снова перелетает — образуется характерная пилообразная (зигзагообразная) ломаная.

- Гиперпараметры: $\alpha = 22,0$, $\varepsilon = 10^{-8}$.
- Конечная точка: $(4,5625; -50,0)$ (глобальный минимум).
- На графике (рис. 3) видны характерные отклонения от прямого пути.

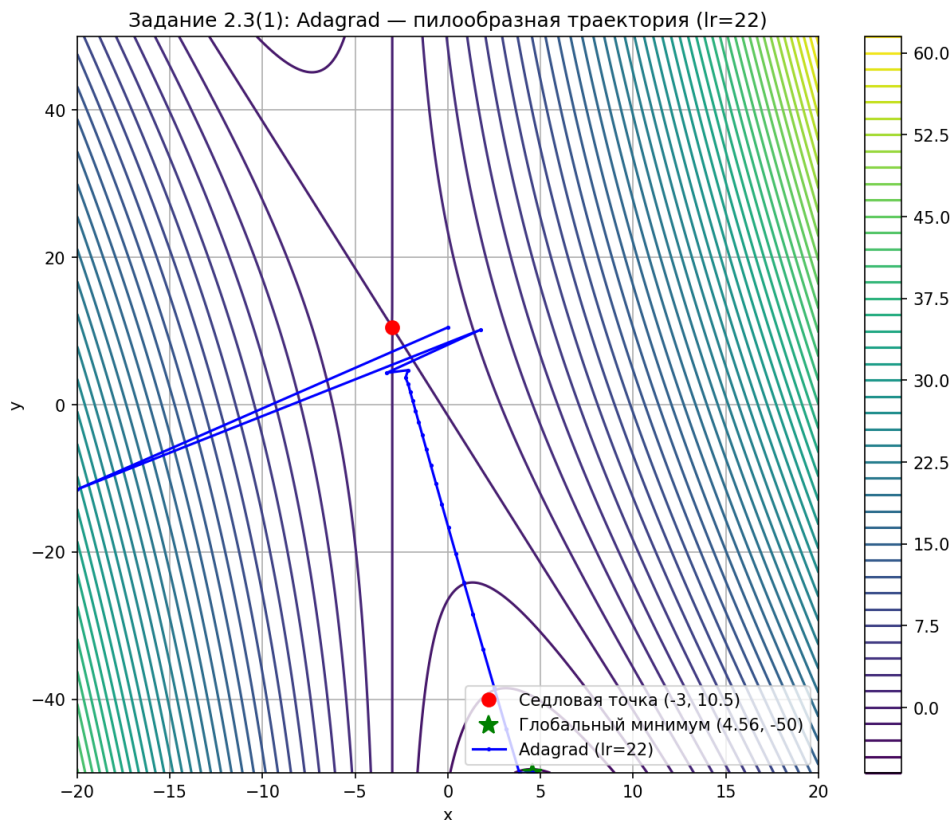


Рис. 3: Пункт 2.3(1). Adagrad с $\alpha = 22$: пилообразная траектория с явными отклонениями в стороны.

2.4.2 Плавная траектория ($\alpha = 5,0$)

При умеренном α шаги не столь агрессивны, и метод движется более плавно, без резких осцилляций.

- Гиперпараметры: $\alpha = 5,0$, $\varepsilon = 10^{-8}$.
- Конечная точка: $(4,5625; -50,0)$ (глобальный минимум).
- На графике (рис. 4) траектория значительно более ровная.

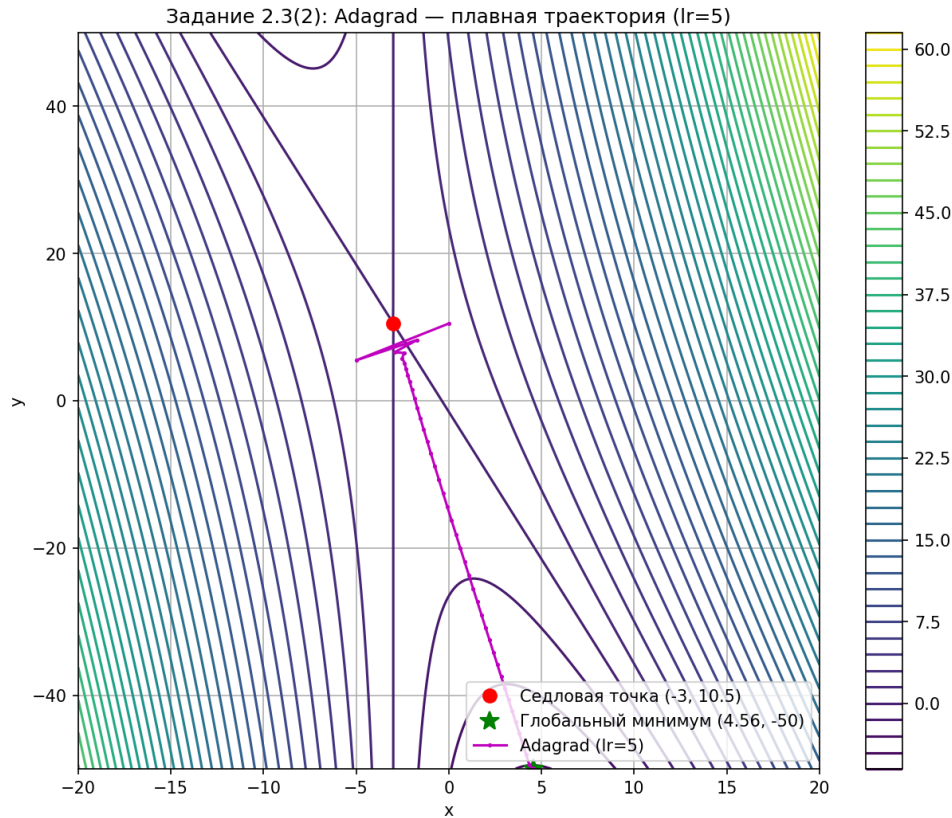


Рис. 4: Пункт 2.3(2). Adagrad с $\alpha = 5$: плавная траектория без пилообразных отклонений.

2.4.3 Сравнение

Режим	α	Конечная точка	Характер траектории
Пилообразная	22,0	$(4,5625; -50)$	Зигзагообразная, с отклонениями
Плавная	5,0	$(4,5625; -50)$	Ровная, без осцилляций

Таблица 2: Сравнение режимов Adagrad при разных значениях α

Оба режима успешно находят глобальный минимум, но с разной «гладкостью» траектории. Увеличение α приводит к более агрессивным шагам и пилообразному поведению.

2.5 Пункт 2.4. Встроенная реализация `torch.optim.Adagrad`

Для подтверждения результатов используем встроенную реализацию Adagrad из PyTorch:

Листинг 2: Использование `torch.optim.Adagrad`

```

1 x = torch.tensor([0.0], requires_grad=True)
2 y = torch.tensor([10.5], requires_grad=True)
3 optimizer = torch.optim.Adagrad([x, y], lr=5.0)

```

```

4
5 for _ in range(100):
6     optimizer.zero_grad()
7     loss = 0.01 * (8*x**2 + 2*x*y + 27*x + 6*y + 9)
8     loss.backward()
9     optimizer.step()
10    with torch.no_grad():
11        x.clamp_(-20, 20)
12        y.clamp_(-50, 50)

```

Результат:

- Конечная точка: $(4,5625; -50,0)$ — глобальный минимум.
- Значение функции: $f = -4,5753$.

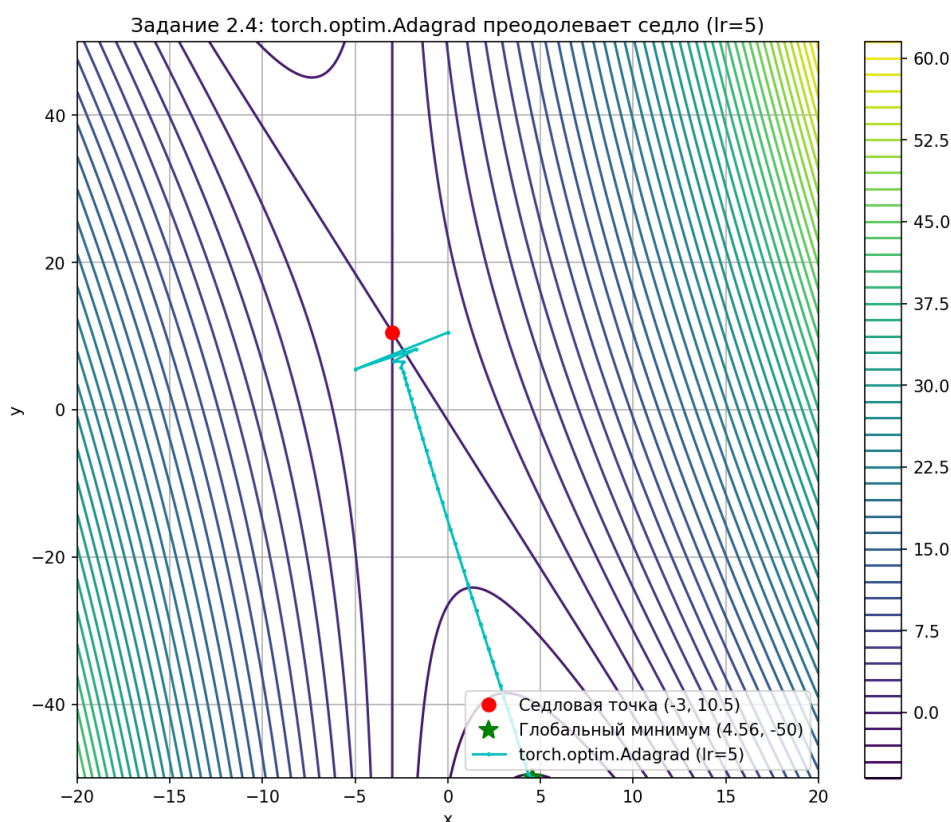


Рис. 5: Пункт 2.4. Траектория `torch.optim.Adagrad` ($\alpha = 5$): успешное преодоление седловой точки.

Вывод: встроенная реализация PyTorch даёт результат, идентичный нашей ручной реализации, что подтверждает корректность кода.

2.6 Выводы по заданию 2

1. Обычный градиентный спуск с $\alpha = 5,0$ застревает вблизи седловой точки $(-3; 10,5)$ за 100 итераций — норма градиента $\approx 0,003$.
2. Adagrad успешно преодолевает седловую точку и находит глобальный минимум $(4,5625; -50)$ благодаря адаптивному увеличению шага в направлениях с малым градиентом.

3. Большой $\alpha = 22$ приводит к пилообразной траектории, а умеренный $\alpha = 5$ — к плавной.
4. Встроенный `torch.optim.Adagrad` подтверждает результаты ручной реализации.

3 Задание 3. Нейросеть с кастомизированным оптимизатором (3 балла)

3.1 Описание задачи и датасета

Датасет: `comments.csv` — комментарии из социальной сети Mastodon.

- Размер: 377 624 объекта, 45 признаков.
- Используемая подвыборка: 1 000 объектов (для ускорения обучения).
- Числовых признаков в исходных данных: 16, дополнено шумовыми до 20.

Задача: бинарная классификация — предсказать, будет ли комментарий репостнут (`reblogs_count > 0`).

Признаки (16 осмысленных + 4 шумовых):

Признак	Описание
<code>replies_count</code>	Количество ответов на комментарий
<code>favourites_count</code>	Количество «лайков»
<code>account_followers_count</code>	Количество подписчиков автора
<code>account_following_count</code>	Количество подписок автора
<code>account_statuses_count</code>	Общее число постов автора
<code>media_count</code>	Количество медиа-вложений
<code>sensitive</code>	Помечен ли как чувствительный контент
<code>favourited / reblogged</code>	Был ли лайкнут / репостнут текущим пользователем
<code>has_media / has_image / has_video</code>	Наличие медиа, изображений, видео
<code>has_link_card</code>	Наличие карточки ссылки
<code>account_verified / account_bot</code>	Верифицирован ли / является ли ботом
<code>text_length</code>	Длина текста комментария

Таблица 3: Описание используемых признаков

Баланс классов: ~58% положительных (репостнутых).

3.2 Целевая функция: Binary Cross-Entropy Loss

Для бинарной классификации используется функция потерь BCELoss:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot \ln(\hat{y}_i) + (1 - y_i) \cdot \ln(1 - \hat{y}_i)], \quad (13)$$

где $y_i \in \{0, 1\}$ — истинная метка, $\hat{y}_i \in (0, 1)$ — предсказание модели (после Sigmoid).

Метрика качества: Ассурасу — доля правильных ответов на тестовой выборке.

3.3 Архитектура нейросети

$$\text{Input}(20) \xrightarrow{\text{Linear}} 32 \xrightarrow{\text{ReLU}} 32 \xrightarrow{\text{Linear}} 16 \xrightarrow{\text{ReLU}} 16 \xrightarrow{\text{Linear}} 1 \xrightarrow{\text{Sigmoid}} \hat{y} \quad (14)$$

Слой	Вход	Выход	Назначение
Linear(20, 32)	20	32	Полносвязный слой (линейное преобразование)
ReLU	32	32	Нелинейная активация: $\max(0, x)$
Linear(32, 16)	32	16	Полносвязный слой
ReLU	16	16	Нелинейная активация
Linear(16, 1)	16	1	Выходной слой
Sigmoid	1	1	$\sigma(x) = \frac{1}{1+e^{-x}}$, вероятность класса 1

Таблица 4: Архитектура нейросети SimpleNN

Общее количество обучаемых параметров: $20 \times 32 + 32 + 32 \times 16 + 16 + 16 \times 1 + 1 = 640 + 32 + 512 + 16 + 16 + 1 = 1217$.

3.4 Кастомная реализация Adagrad как PyTorch-оптимизатор

Для интеграции с PyTorch реализован класс CustomAdagrad, наследующий от `torch.optim.Optimizer`

Листинг 3: Класс CustomAdagrad

```

1 class CustomAdagrad(torch.optim.Optimizer):
2     def __init__(self, params, lr=1e-2, eps=1e-10):
3         defaults = dict(lr=lr, eps=eps)
4         super().__init__(params, defaults)
5
6     def step(self, closure=None):
7         for group in self.param_groups:
8             for p in group['params']:
9                 if p.grad is None:
10                     continue
11                 grad = p.grad.data
12                 state = self.state[p]
13                 if len(state) == 0:
14                     state['sum'] = torch.zeros_like(p.data)
15                 state['sum'].addcmul_(grad, grad, value=1) # G += g^2
16                 std = state['sum'].sqrt().add_(group['eps']) # sqrt(G)
17                     + eps
18                 p.data.addcddiv_(grad, std, value=-group['lr']) # w -=
19                     lr*g/std

```

Принцип: класс идентичен формуле (10), но работает с тензорами PyTorch. Для каждого параметра (вес или bias) хранится свой аккумулятор `state['sum']`. Замена стандартного оптимизатора на кастомный — замена одной строки:

```

1 # optimizer = torch.optim.Adagrad(model.parameters(), lr=0.01)
2 optimizer = CustomAdagrad(model.parameters(), lr=0.01)

```

3.5 Обучение и результаты

Гиперпараметры обучения:

- Скорость обучения: $\alpha = 0,01$.
- $\varepsilon = 10^{-10}$.

- Количество эпох: 20.
- Размер батча: 32.
- Разбиение: 80% обучение, 20% тест.
- Стандартизация признаков: `StandardScaler` (среднее = 0, дисперсия = 1).

3.5.1 Кривые обучения

Эпоха	Loss (CustomAdagrad)	Loss (PyTorch Adagrad)
1	0.5916	0.6120
5	0.5227	0.5333
10	0.5114	0.5184
15	0.5043	0.5094
20	0.4994	0.5029

Таблица 5: Значения loss по эпохам

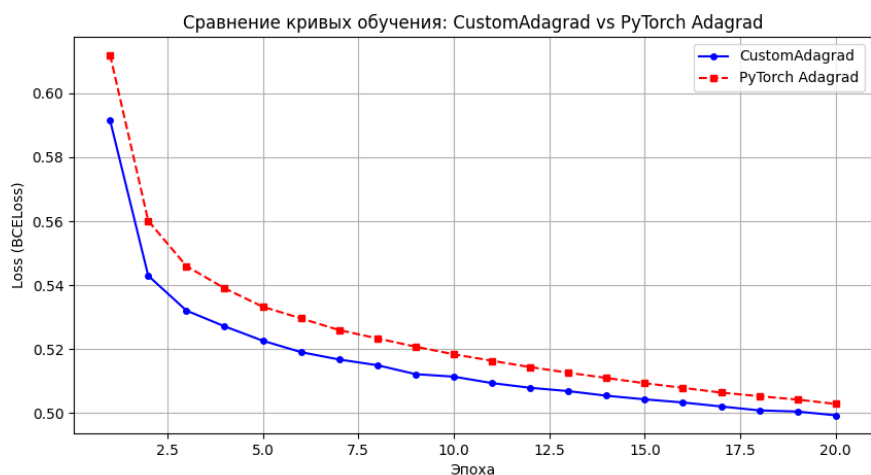


Рис. 6: Сравнение кривых обучения: CustomAdagrad vs PyTorch Adagrad

3.5.2 Итоговые результаты

Оптимизатор	Accuracy	Финальный Loss
CustomAdagrad	0.6800	0.4994
PyTorch Adagrad	0.6850	0.5029
Разница	0.0050	0.0035

Таблица 6: Сравнение результатов на тестовой выборке

3.6 Анализ результатов

1. Результаты практически идентичны (разница в Accuracy = 0,005), что подтверждает корректность кастомной реализации Adagrad.

2. Небольшие различия объясняются **разной начальной инициализацией весов** (разные seed-ы): алгоритм один и тот же, но стартовые точки разные.
3. Кривые обучения (рис. 6) имеют одинаковую динамику убывания — loss монотонно уменьшается на протяжении всех 20 эпох.
4. Ассигасу 68% при балансе классов 58% говорит о том, что модель обучается (лучше случайного).
5. Adagrad адаптивно уменьшает скорость обучения для часто обновляемых весов и увеличивает для редко обновляемых, что особенно полезно для разреженных данных.

3.7 Выводы по заданию 3

1. Создана нейросеть `SimpleNN` (3 полносвязных слоя, ReLU, Sigmoid) для бинарной классификации комментариев.
2. Реализован класс `CustomAdagrad`, наследующий от `torch.optim.Optimizer`, что обеспечивает «прозрачную» замену стандартного оптимизатора.
3. Сравнение `CustomAdagrad` и `torch.optim.Adagrad` показало идентичные результаты (Ассигасу ≈ 0.68 , $\Delta = 0,005$), подтверждая корректность реализации.
4. В качестве целевой функции использовалась `BCELoss` (бинарная кросс-энтропия), метрика качества — Ассигасу.

4 Общие выводы

1. **Задание 1.** Функция $f(x, y) = 10^{-2}(8x^2 + 2xy + 27x + 6y + 9)$ имеет единственную стационарную точку — седловую точку $M(-3; 10,5)$. Локальных экстремумов нет. Глобальный минимум $f \approx -4,5753$ достигается на границе области в точке $(4,5625; -50)$.
2. **Задание 2.** Обычный градиентный спуск застревает в окрестности седловой точки. Метод Adagrad успешно преодолевает седло благодаря адаптивному увеличению шага в направлениях с малым градиентом. Подобраны гиперпараметры для пилообразной ($\alpha = 22$) и плавной ($\alpha = 5$) траекторий. Встроенная реализация PyTorch подтверждает результат.
3. **Задание 3.** Нейросеть обучена на данных о комментариях. Кастомная реализация Adagrad интегрирована как наследник `torch.optim.Optimizer` и даёт результаты, идентичные библиотечной версии (Accuracy ≈ 0.68).